

Ri-acquisizione del corpus Dataverse — Runbook del pilota (Fase 1)

Direzione ICT, Università degli Studi di Milano — Progetto Arkive

Luglio 2026

Indice

Runbook del pilota — Fase 1	1
1. Scopo e contesto	1
2. Esito della Fase 0 (dati di partenza)	2
3. Dataset pilota selezionati	2
4. Prerequisiti (da verificare PRIMA di iniziare)	2
5. Esecuzione	3
5.1 Camera pulita — DENTRO la Transfer Source Location	3
5.2 Lista DOI del pilota	4
5.3 Download	4
6. Gate 1 — Validazione dell’acquisizione	4
7. Ingest e Gate 2 — Validazione dell’AIP	6
7.1 Ingest	6
7.2 Gate 2 — controlli sull’AIP	6
8. I 6 DOI non più raggiungibili	8
9. Blocchi durante l’ingest — diagnosi e rimedi	8
9.1 Transfer fermo in <code>USER_INPUT</code>	8
9.2 Nessun job avanza: MCPClient disallineato	8
9.3 Lo script riaggancia un transfer annullato	9
9.4 Procedura completa di ripartenza pulita	9
10. Politica di trattamento degli archivi compressi	9
10.1 Il problema	10
10.2 Verifica empirica	10
10.3 Verifica del doppio passaggio sui <code>.tar.gz</code>	10
10.4 Decisione adottata: configurazione differenziata a due lotti	11
10.5 Applicazione e verifica	11
11. Export dei metadati per versione: limite noto dell’istanza	11
12. Piano dei lotti e calibrazione	12
12.1 Dati di calibrazione misurati	12
13. Criteri di GO / NO-GO per il lotto completo	12
14. Registrazione dell’esito	13
14.1 Esito del pilota del 22 luglio 2026	13
14.2 Esito della calibrazione sul primo lotto reale (<code>LOTTO_27</code>)	13

Runbook del pilota — Fase 1

1. Scopo e contesto

Il corpus Dataverse è stato acquisito e ingerito in Archivematica **prima** che la pipeline introducesse due comportamenti oggi ritenuti necessari:

1. la **verifica di integrità** dei file scaricati (checksum/dimensione), che impedisce a un download troncato di entrare silenziosamente nel SIP;
2. la **conservazione dell'originale** per i file ingeriti come tabellari (`format=original`), invece della derivata `.tab` generata da Dataverse.

Poiché gli AIP prodotti finora sono **sperimentali** (progetto in fase avanzata ma non ancora in produzione), è stato deciso di **non migrare** i pacchetti esistenti ma di **ri-acquisire l'intero corpus** con la pipeline corretta e ricostruire gli AIP da una baseline pulita. Questo evita di trascinare nella futura produzione catene di `supersede` e record di deaccession relativi a materiale di prova, e — cosa più importante — sottopone a verifica di integrità **tutto** il corpus, non solo i dataset con l'anomalia nota.

Questo documento descrive la **Fase 1**: l'esecuzione controllata del ciclo completo su **due dataset pilota**, allo scopo di validare la procedura prima di applicarla ai 702 dataset dello scope. Il prodotto della Fase 1 non è solo “due pacchetti corretti”, ma la **procedura verificata** da ripetere sul lotto.

2. Esito della Fase 0 (dati di partenza)

La ricognizione con `verifica_raggiungibilita.py` sui 708 DOI dello scope ha dato:

Indicatore	Valore
DOI ri-acquisibili (OK)	672
DOI vuoti (nessun file in nessuna versione)	30
DOI non più raggiungibili su Dataverse	6
DOI con file ad accesso ristretto	111
Versioni totali da scaricare	1 226
File totali da scaricare	67 730
Volume stimato (originali, tutte le versioni)	62,95 GB

I 6 DOI non più raggiungibili — QA4BR0, BX8AIN, YF2NZV, WKMYTV, YIAURF, OYDM83 — sono **fuori dallo scope della ri-acquisizione** e vanno trattati a mano: vedi sezione 8.

3. Dataset pilota selezionati

I due pilota sono scelti per **esercitare i percorsi nuovi**, non per comodità:

Pilota	DOI	Versioni	File	Ingeriti	Ristretti	Volume
A	doi:10.13130/RD_UNIMI/PKZ7FJ	4	10	8	0	4,3 MB
B	doi:10.13130/RD_UNIMI/DP08SC	3	26	23	25	1,1 MB

Perché A: quattro versioni e otto file ingeriti su dieci. Valida in un colpo solo la struttura per versione (`objects/<versione>/objects/`), il download `format=original` e l'identificazione dei formati reali, restando piccolo.

Perché B: quasi tutti i file sono **contemporaneamente ristretti e ingeriti** (25 ristretti, 23 ingeriti su 26). È l'intersezione più rischiosa dell'intero corpus: richiede che `format=original` funzioni **su file ad accesso ristretto con API key**. Se in quella combinazione mancasse un permesso, il pacchetto risulterebbe silenziosamente incompleto su una parte significativa dei 111 DOI ristretti.

4. Prerequisiti (da verificare PRIMA di iniziare)

Nessuno di questi punti è formale: ognuno corrisponde a un incidente già occorso.

4.1 Sincronizzazione Dropbox disattivata sulla cartella di lavoro. Per il pilota è sufficiente la sospensione della sincronizzazione. Per il lotto completo va valutata la Selective Sync o lo spostamento della cartella fuori da Dropbox (vedi sezione 9).

4.2 Mount drvfs con opzione metadata. Senza di essa `utime()` fallisce e lo store dell'AIP termina con errore `rsync` codice 23 anche quando il trasferimento dei file è riuscito. Verifica:

```
mount | grep drvfs
# atteso: ... type drvfs (rw,noatime,...,metadata,uid=...,gid=...,umask=022)
cat /etc/wsl.conf
```

4.3 Spazio disponibile. Per il pilota bastano pochi MB, ma è il momento giusto per misurare l'headroom in vista del lotto (63 GB di download più lavorazione Archivematica e AIP risultanti):

```
df -h /mnt/e
```

4.4 File .env completo, con `DATAVERSE_API_KEY` valorizzata: **indispensabile** per il Pilota B (file ristretti). Verifica rapida senza stampare la chiave:

```
cd tools/dataverse-archivematica
grep -c DATAVERSE_API_KEY .env # atteso: 1
```

4.5 Utente di esecuzione. Gli script della pipeline **non vanno mai eseguiti come utente archivematica**. Verifica con `whoami`.

4.6 Versione degli script. Il pilota ha senso solo con la pipeline aggiornata:

```
git log --oneline -1
grep -c "_is_ingested_tabular" scarica_dataverse.py # atteso: >= 2
grep -c "doi_path.resolve()" archivematica_ingest.py # atteso: 1
```

4.7 Processing configuration corretta — preconditione critica. Due scelte di `dataverse_001` determinano la qualità archivistica degli AIP:

Decisione	Valore richiesto
Perform file format identification (Transfer)	Yes
Perform file format identification (Ingest)	<i>No, use existing data</i>
Normalize	Normalize for preservation
Approve normalization	Yes

Senza identificazione in fase di transfer, il METS riporta **Unknown** per **tutti** gli oggetti: niente PUID, niente pianificazione della conservazione, e la normalizzazione non può funzionare (il FPR sceglie le regole in base al formato riconosciuto). L'impostazione "*No, use existing data*" in ingest è corretta solo se l'identificazione in transfer è attiva, altrimenti non esistono dati da riusare.

Il file **non va modificato a mano**: gli UUID delle catene variano tra versioni di Archivematica. Si genera dal Dashboard (*Administration -> Processing configuration -> dataverse_001 -> Edit -> Save -> Download*) e si copia nella posizione della copia locale versionata. Verifica:

```
CFG=~/.arkive/config/dataverse_001ProcessingMCP.xml
grep -A2 "f09847c2-ee51-429a-9478-a860477f6b8d" $CFG # identificazione: NON "Skip"
grep -A2 "cb8e5706-e73f-472f-ad9b-d1236af8095f" $CFG # normalizzazione
grep -A2 "7509e7dc-1e1b-4dce-8d21-e130515fce73" $CFG # normalizzazione (2° punto)
```

5. Esecuzione

5.1 Camera pulita — DENTRO la Transfer Source Location

La ri-acquisizione **non** si sovrappone a cartelle esistenti: la pipeline salva ora i nomi originali (`.csv`, `.dta`), quindi su una cartella popolata in passato scaricherebbe gli originali **accanto** ai vecchi `.tab`, lasciando

entrambi.

La cartella di lavoro deve inoltre trovarsi **fisicamente dentro il percorso della Transfer Source Location** dello Storage Service. Motivo: `archivematica_ingest.py` avvia il transfer passando `base64(<location_uuid>:<path_a` e lo Storage Service accetta solo percorsi interni alla location. Un pacchetto fuori da lì non verrebbe visto da Archivematica.

Allo stesso tempo la cartella deve essere una **sottocartella dedicata**, non la radice della Transfer Source: se si omette `--source`, lo script raccoglie *tutti* i pacchetti DOI presenti nella location, compresi quelli della vecchia acquisizione, che il pilota non deve toccare.

```
# 1) ricavare il percorso della Transfer Source Location (o usare quello noto)
TS_PATH=/percorso/della/transfer_source      # adattare
```

```
# 2) creare la sottocartella dedicata al pilota, vuota
mkdir -p "$TS_PATH/PILOTA"
ls -A "$TS_PATH/PILOTA"      # deve essere vuota
```

Se il download è già stato eseguito altrove, spostare i pacchetti (non copiarli, per non lasciare duplicati):

```
mv PILOTA/doi_* "$TS_PATH/PILOTA"/
```

5.2 Lista DOI del pilota

```
cat > dois_pilota.txt << 'EOF'
doi:10.13130/RD_UNIMI/PKZ7FJ
doi:10.13130/RD_UNIMI/DP08SC
EOF
```

5.3 Download

```
python3 scarica_dataverse.py --dois dois_pilota.txt --output "$TS_PATH/PILOTA" --dcat \
2>&1 | tee logs/pilota_download.log
```

--dcat è necessario: il pilota deve validare anche che la `dcat:distribution` descriva l'originale.

6. Gate 1 — Validazione dell'acquisizione

Non procedere all'ingest se anche uno solo di questi controlli fallisce.

6.1 Nessun errore di download. Nel `riepilogo_download.json` entrambi i DOI devono risultare ok, mai partial:

```
python3 -c "
import json,os; TS=os.environ['TS_PATH']
d=json.load(open(TS+'/PILOTA/riepilogo_download.json'))
print(json.dumps(d, indent=2)[:800])
"
grep -c "ERRORE\\|[RISCARICO]" logs/pilota_download.log      # atteso: 0
```

6.2 Nessun fallback alla derivata. Se compare `[FALLBACK]`, per quel file `format=original` non ha risposto e nel pacchetto è finita la `.tab`:

```
grep "\[FALLBACK\]" logs/pilota_download.log      # atteso: nessun risultato
```

Controllo di contabilità speculare, che conferma il caso positivo:

```
grep -c "Scarico:" logs/pilota_download.log      # atteso: 36 (10 + 26)
grep -c "originale di" logs/pilota_download.log  # atteso: 31 (8 + 23)
```

Il primo è il totale dei file scaricati, il secondo quelli scaricati **come originale** al posto della derivata (etichetta nome.csv (originale di nome.tab)). La differenza (36 – 31 = 5) deve corrispondere ai file **non** ingeriti, cioè privi di un originale distinto. Se la contabilità non chiude, qualche file ingerito non è stato trattato come tale e va indagato prima di procedere.

6.3 Sono stati conservati gli ORIGINALI, non i .tab. È il controllo centrale del Livello 2:

```
find "$TS_PATH/PILOTA" -name "*.tab" | head           # atteso: NESSUN risultato
find "$TS_PATH/PILOTA" -path "*/objects/*" -type f -not -path "*/metadata/*" | head -20
```

I nomi vanno preservati esattamente come depositati. Nell'elenco è normale — e corretto — trovare nomi con spazi (IVD_NMBU shipment 2_cytotoxicity_NRU.xlsx) o con estensioni anomale ripetute (..._ANPEP.xlsx.xlsx). La pipeline usa `originalFileName` così com'è: quei nomi sono letteralmente quelli con cui il ricercatore ha depositato i file. Conservarli invariati, refusi compresi, è il comportamento corretto ai fini dell'autenticità dell'oggetto archiviato; una normalizzazione silenziosa produrrebbe oggetti diversi dagli originali.

6.4 Struttura per versione completa. Il Pilota A deve avere 4 versioni, il Pilota B 3:

```
ls -d "$TS_PATH"/PILOTA/*PKZ7FJ*/objects/*/          # atteso: 4 directory
ls -d "$TS_PATH"/PILOTA/*DP08SC*/objects/*/          # atteso: 3 directory
ls "$TS_PATH"/PILOTA/*DP08SC*/objects/*/metadata/
```

Asimmetria attesa nei metadati di versione: `dataverse.json`, `dcat.json` e `metadata.csv` sono presenti in **ogni** versione perché generati localmente dalla pipeline; `schema.json` compare **solo nell'ultima versione**, perché è scaricato dall'exporter Schema.org di Dataverse, che lavora a livello di dataset e non espone le versioni storiche. La sua assenza nelle versioni precedenti **non è un errore di export**. Da verificare invece se mancasse in modo irregolare (per esempio assente anche nell'ultima versione, o presente a intermittenza).

6.5 I file ristretti sono stati scaricati (Pilota B). Contro il rischio del pacchetto silenziosamente incompleto: il numero di file di dati presenti su disco deve corrispondere a quello atteso dal report di Fase 0.

```
find "$TS_PATH"/PILOTA/*DP08SC* -path "*/objects/*" -type f \
-not -path "*/metadata/*" | wc -l           # atteso: 26
```

L'esclusione di `*/metadata/*` è necessaria: nella struttura `objects/<versione>/metadata/` la parola `objects` compare comunque nel percorso, quindi senza il filtro verrebbero conteggiati anche `dcat.json`, `metadata.csv`, `dataverse.json` e `schema.json`, falsando il totale.

6.6 Il DCAT descrive l'originale.

```
python3 -c "
import json,glob,os
TS=os.environ['TS_PATH']
for p in sorted(glob.glob(TS+'*/PILOTA*/objects/*/metadata/dcat.json'))[:2]:
    d=json.load(open(p)); dd=d.get('dcat:distribution')
    dd=dd if isinstance(dd,list) else [dd]
    print(p)
    for x in dd[:3]:
        print(' ', x.get('dct:title'), '|', x.get('dcat:mediaType'), '|',
              x.get('dcat:accessURL',{}).get('@id','')[-30:])
"
```

Atteso: titoli con estensione originale (.csv, .xlsx, .dta), `mediaType` dell'originale (**non** text/tab-separated-values) e `accessURL` che termina con `?format=original`.

Attenzione — è normale che alcune `accessURL` NON abbiano `?format=original`. La pipeline decide file per file: i file **non ingeriti** da Dataverse (privi dei campi `original*`) non hanno una rappresentazione originale distinta, quindi il loro `accessURL` è quello semplice. Aggiungere `?format=original` significherebbe descrivere una rappresentazione inesistente. Un .xlsx può benissimo non essere stato ingerito (fogli multipli, dimensione, ingest non riuscito): non è un errore.

Verifica di contabilità sull'intero pilota, che chiude il cerchio con il log:

```
python3 - << 'EOF'
import json, glob, os
TS=os.environ['TS_PATH']
tot=orig=plain=0
for p in sorted(glob.glob(TS+'*/PILOTA/*/objects/*/metadata/dcat.json')):
    d=json.load(open(p)); dd=d.get('dcat:distribution') or []
    dd=dd if isinstance(dd,list) else [dd]
    for x in dd:
        tot+=1
        u=x.get('dcat:accessURL',{}).get('@id','')
        if u.endswith('?format=original'): orig+=1
        else: plain+=1
        if 'tab-separated' in (x.get('dcat:mediaType') or ''):
            print('!! ANOMALIA (mediaType .tab):', p, x.get('dct:title'))
print(f"distribution totali: {tot} con format=original: {orig} senza: {plain}")
EOF
```

Atteso: **36 totali, 31 con format=original, 5 senza** — gli stessi numeri dei `grep` del punto 6.2. Nessuna riga di anomalia. In caso di dubbio su un singolo file, la conferma si legge in `dataverse.json`: se `originalFileName` e `originalFileFormat` sono `None`, il file non è ingerito e l'URL semplice è corretto.

6.7 metadata.csv coerente. La colonna `filename` è costruita leggendo il disco, quindi deve elencare i nomi originali:

```
head -3 "$TS_PATH"/PILOTA/*PKZ7FJ*/metadata/metadata.csv
```

7. Ingest e Gate 2 — Validazione dell'AIP

7.1 Ingest

```
python3 archivematica_ingest.py --source "$TS_PATH/PILOTA" --auto-approve \
--state-file stato_pilota.json 2>&1 | tee logs/pilota_ingest.log
```

Due accorgimenti, entrambi necessari:

- **--source esplicito.** Omettendolo, lo script userebbe la radice della Transfer Source Location e raccoglierebbe *tutti* i pacchetti DOI presenti, inclusi quelli della vecchia acquisizione: il pilota ingerirebbe molto più del previsto. Puntando alla sottocartella si lavora solo sui due dataset.
- **File di stato separato** (`stato_pilota.json`), per non mescolare il pilota con lo storico di produzione in `stato_archivematica.json`.

Prima di lanciare, verificare che nella sottocartella ci siano esattamente i due pacchetti attesi:

```
ls -ld "$TS_PATH/PILOTA"/doi_*          # atteso: 2 directory
```

7.2 Gate 2 — controlli sull'AIP

7.2.1 AIP presente e memorizzato. Annotare gli UUID e verificare lo stato:

```
python3 - << 'EOF'
import os, requests, urllib3
urllib3.disable_warnings()
for l in open('.env'):
    l=l.strip()
    if l and not l.startswith('#') and '=' in l:
        k,_,v=l.partition('='); os.environ.setdefault(k.strip(), v.strip().strip('"\''))
SS=os.environ.get('SS_URL','http://localhost:8000')
```

```
H={'Authorization': f"ApiKey {os.environ['SS_USER']}:{os.environ['SS_API_KEY']}"}
for nome,uuid in [('DP08SC','<AIP-UUID>'), ('PKZ7FJ','<AIP-UUID>')]:
    d=requests.get(f"{SS}/api/v2/file/{uuid}/",headers=H,verify=False,timeout=60).json()
    print(f"{nome}: status={d.get('status')} size={d.get('size')}")
    print(f"    {d.get('current_full_path')}")
EOF
```

Atteso: **status=UPLOADED**. Le dimensioni devono essere coerenti con il volume degli originali stimato in Fase 0.

7.2.2 Formati, eventi e agenti — il controllo decisivo. Legge il METS dall'AIP compresso senza estrarlo:

```
python3 - << 'EOF'
import tarfile, glob, re
from collections import Counter
import xml.etree.ElementTree as ET
for uuid, nome in [('<primi8>','DP08SC'), ('<primi8>','PKZ7FJ')]:
    aip = glob.glob(f"/mnt/e/ARKIVE/AIPsStore/{uuid[:4]}/**/*{uuid}*.tar.gz", recursive=True)
    with tarfile.open(aip[0]) as t:
        mets=[m for m in t.getnames() if re.search(r'/data/METS/.*\.xml$', m)]
        xml=t.extractfile(mets[0]).read()
        root=ET.fromstring(xml); tag=lambda e: e.tag.split(' ')[-1]
        print(f"\n=== {nome}")
        for f,n in Counter(e.text for e in root.iter() if tag(e)=='formatName').most_common():
            print(f"    {n:4d} {f}")
        for e,n in Counter(x.text for x in root.iter() if tag(x)=='eventType').most_common():
            print(f"    [ev] {n:4d} {e}")
EOF
```

Attesi e criteri di lettura:

- i formati devono essere **identificati**: Microsoft Excel, CSV, Plain Text, JSON Data Interchange Format... e **nessun Unknown**;
- **nessun Tab-Separated Values** tra gli oggetti: se comparisse, nell'AIP è finita una derivata invece dell'originale;
- deve esserci un evento **format identification** per ogni oggetto;
- gli eventi **normalization** compaiono **solo per i formati con una regola FPR** (tipicamente PDF, immagini, audio, video). La loro assenza sugli .xlsx non è un guasto: il FPR considera quei formati già adeguati alla conservazione;
- l'agente e la versione dello strumento di identificazione si leggono nell'**eventDetail** (es. **program="Siegfried"**; **version="1.11.2"**).

7.2.3 Conteggio degli oggetti — attenzione a cosa si conta. Il numero di file nel METS è **maggiore** di quello dei file di dati, perché i metadati per versione (**objects/vX.Y/metadata/...**) stanno dentro **objects/** e Archivematica li tratta come contenuto. Per DP08SC: 26 file di dati + 10 metadati = 36; per PKZ7FJ: 10 + 13 = 23. Solo la cartella **metadata/** di primo livello è trattata come metadati descrittivi.

7.2.4 Nomi originali preservati. Archivematica **sanifica** i nomi con spazi o caratteri problematici sul filesystem e registra ogni modifica con un evento **filename change**. Il nome di deposito resta documentato nel METS in **originalName**. Verifica che gli spazi ci siano ancora:

```
python3 - << 'EOF'
import tarfile, glob, re
import xml.etree.ElementTree as ET
uuid='<primi8>'
aip=glob.glob(f"/mnt/e/ARKIVE/AIPsStore/{uuid[:4]}/**/*{uuid}*.tar.gz", recursive=True)
with tarfile.open(aip[0]) as t:
```

```

    mets=[m for m in t.getnames() if re.search(r'/data/METS\..*\.xml$', m)]
    xml=t.extractfile(mets[0]).read()
    root=ET.fromstring(xml); tag=lambda e: e.tag.split(' ')[-1]
    orig=[e.text for e in root.iter() if tag(e)=='originalName' and e.text and not e.text.endswith('/')]
    print("con spazi nel nome:", sum(1 for o in orig if ' ' in o.split('/')[1]), "su", len(orig))
EOF

```

7.2.5 Fixity. Nessun evento PREMIS di *fixity check* fallito.

8. I 6 DOI non più raggiungibili

QA4BR0, BX8AIN, YF2NZV, WKMYTV, YIAURF, OYDM83 non esistono più su `dataverse.unimi.it` (verificato via API, browser e curl). Per questi la ri-acquisizione è **impossibile**, quindi:

1. **Prima di qualunque pulizia**, verificare se ne esiste una copia locale (nelle cartelle di download o negli AIP già prodotti) e, in caso affermativo, **metterla al sicuro fuori dall'area di lavoro**: sarebbe l'unico esemplare rimasto.
2. **Segnalare la scomparsa ai responsabili di Data@UNIMI**. Sei dataset pubblicati con DOI assegnato che spariscono da un repository certificato CoreTrustSeal richiedono una spiegazione (deaccession su richiesta dell'autore, DOI di test rimossi, o altro), che va documentata.
3. Non includerli nella lista del lotto.

9. Blocchi durante l'ingest — diagnosi e rimedi

Questi casi si sono presentati tutti durante la calibrazione del primo lotto e sono destinati a ripresentarsi su 27 lotti. Vanno riconosciuti in fretta perché, a colpo d'occhio, si somigliano molto: lo script attende, il transfer risulta `PROCESSING`, e apparentemente “non succede nulla”.

9.1 Transfer fermo in `USER_INPUT`

Archivematica ha incontrato una decisione **non coperta** dalla `processingMCP.xml` e attende una risposta. L'opzione `--auto-approve` gestisce solo l'approvazione iniziale del transfer, non le scelte successive.

Il caso riscontrato: un dataset composto da un unico archivio compresso ha fatto comparire la domanda **“Extract packages?”**, assente dalla configurazione. La soluzione non è rispondere a mano — su 672 pacchetti si ripresenterebbe decine di volte — ma **aggiungere la scelta alla processing configuration** e rilanciare.

Impostazione adottata: **Extract packages = Yes, Delete packages after extraction = No**. In questo modo l'AIP contiene sia l'archivio originale come depositato (autenticità) sia il contenuto estratto, identificato, verificato e normalizzato (conservazione effettiva). Conservare un archivio non estratto significherebbe custodire un oggetto opaco: nessuna identificazione dei formati interni, nessun monitoraggio dell'obsolescenza, nessuna normalizzazione possibile.

Il `processingMCP.xml` viene letto all'AVVIO del transfer. Aggiornare la configurazione mentre un transfer è in corso non ha alcun effetto su quel transfer: va annullato e riavviato. Verificare inoltre che il file **dentro il pacchetto** sia quello nuovo (lo script lo ricopia a ogni esecuzione, ma un transfer già avviato usa la copia letta in partenza).

9.2 Nessun job avanza: MCPClient disallineato

Sintomo: `[approve]` Nessun transfer in attesa trovato ripetuto, poi `[transfer]` stato: `PROCESSING` all'infinito. In Dashboard il transfer non progredisce.

Causa: dopo aver **fermato e riavviato MCPServer** (operazione necessaria per pulire le watched directory), MCPClient resta agganciato alla sessione Gearman precedente e smette di prelevare i job. Il server li accoda, nessuno li esegue.

L'insidia è che tutti i servizi risultano **active**: il segnale vero è il **disallineamento degli orari di avvio** e l'assenza di righe recenti ***** RUNNING TASK: ... ***** nel log del client.

```
systemctl status archivematica-mcp-server archivematica-mcp-client gearman-job-server
sudo journalctl -u archivematica-mcp-client -n 30 --no-pager
```

Regola operativa: dopo ogni riavvio di MCPServer, riavviare anche MCPClient.

```
sudo systemctl start archivematica-mcp-server
sudo systemctl restart archivematica-mcp-client
```

9.3 Lo script riaggancia un transfer annullato

Sintomo: nel log compare **Transfer UUID (ripresa): <uuid>** seguito da tentativi di approvazione che non trovano mai nulla. Lo script non sta avviando un transfer nuovo: sta cercando di riprendere quello registrato nel file di stato, che nel frattempo è stato annullato dalla Dashboard e quindi non comparirà mai tra i transfer in attesa.

È il rovescio del meccanismo di ripresa (`find_transfer_uuid_by_name`), utile dopo un timeout di rete ma dannoso quando il transfer è stato eliminato di proposito.

Rimedio: annullando un transfer dalla Dashboard, rimuovere anche la voce corrispondente dal file di stato, così il DOI riparte da zero.

```
cp stato_riacquisizione.json stato_riacquisizione.json.bak
python3 - << 'EOF'
import json
p='stato_riacquisizione.json'; d=json.load(open(p))
k='doi_10.13130_RD_UNIMI_XXXXXX'      # adattare
if k in d:
    print("rimuovo:", d.pop(k)); json.dump(d, open(p,'w'), indent=2, ensure_ascii=False)
print("voci rimaste:", len(d))
EOF
```

Al rilancio il log deve mostrare **Avvio transfer '...' con un UUID nuovo**, non più **Transfer UUID (ripresa)**.

9.4 Procedura completa di ripartenza pulita

Quando un pacchetto va rifatto da zero, l'ordine conta:

1. interrompere lo script (Ctrl-C: lo stato è già su disco);
2. annullare il transfer dalla Dashboard;
3. `sudo systemctl stop archivematica-mcp-server`;
4. rimuovere le copie residue del pacchetto dalle watched directory (`sudo find /var/archivematica/sharedDirectory/watched -maxdepth 3 -name "*<DOI>*"`);
5. `sudo systemctl start archivematica-mcp-server` e `sudo systemctl restart archivematica-mcp-client`;
6. rimuovere la voce del DOI dal file di stato (9.3);
7. verificare che il `processingMCP.xml` nel pacchetto sia quello aggiornato;
8. rilanciare lo stesso comando di ingest.

10. Politica di trattamento degli archivi compressi

Questa sezione documenta una **decisione di politica di conservazione**, presa durante il pilota sulla base di verifiche empiriche. Va conservata come tale: non è un dettaglio di configurazione, ma una scelta che determina la forma degli oggetti conservati e che va poter essere motivata in sede di audit.

10.1 Il problema

Con `Extract packages = Yes` Archivematica estrae gli archivi e tratta ogni file interno come oggetto autonomo. La scelta è stata attivata perché conservare un archivio non estratto significa custodire un oggetto **opaco**: nessuna identificazione dei formati interni, nessun monitoraggio dell'obsolescenza, nessuna normalizzazione possibile.

Il censimento dello scope (`censisci_archivi.py`) ha però mostrato che i file riconosciuti come “archivio” appartengono a **due categorie molto diverse**:

Categoria	Esempi	L'estrazione è...
Contentori veri	FIBRE.zip, Output.zip, MARBLE.rar, qcr_instances.tgz	utile : dentro c'è una gerarchia di file identificabili
File singoli compressi	*.rds (serializzazioni R), *.Rdata, *.fastq.gz, *.sql.gz	dannosa : produce un blob non utilizzabile

Un `.rds` non è un contenitore: è il formato nativo di R, compresso con gzip. Dataverse ne dichiara il `contentType` come `application/gzip` e Siegfried lo identifica come **GZip (PUID x-fmt/266)**, quindi la regola di estrazione del FPR si attiva.

10.2 Verifica empirica

Test su `doi:10.13130/RD_UNIMI/HADWPV` (3 file, di cui un `.rds` da 814 KB), METS dell'AIP risultante:

- **1 evento unpacking** → il `.rds` è stato effettivamente estratto;
- tra i formati compare **1 Unknown** → il contenuto estratto è la serializzazione R grezza, che nessun registro di formati riconosce;
- oggetti totali 11 anziché 10.

L'estrazione produce quindi **zero guadagno di conservazione**: da un `.rds` valido esce un blob inservibile senza il wrapper. L'originale resta (grazie a `Delete packages after extraction = No`), ma accanto vi è un duplicato inutile.

Impatto sullo scope: i dataset a base `.rds` sommano circa **11 500 file** (NZRX1C 6 615, PLHGBP 1 620, UGSD3F 1 620, UZ25HQ 1 080, V0TP76 285, 1MZNNS 180 e altri). L'estrazione ne genererebbe circa **23 000** anziché 11 500, con il relativo tempo di processing e nessun beneficio. Sui `.fastq.gz` si aggiungerebbe una forte crescita di spazio, dato che il formato compresso è la forma standard di distribuzione dei dati di sequenziamento.

10.3 Verifica del doppio passaggio sui `.tar.gz`

Prima di decidere è stato accertato **come Archivematica tratta i veri archivi `.tar.gz`**, perché da questo dipende la portata della scelta.

Siegfried non distingue i due casi. Su un `.tar.gz` contenente una directory e su un banale `.txt.gz`, l'identificazione è identica:

```
id: x-fmt/266   format: GZIP Format   basis: extension match gz; byte match at 0, 3
```

È una conseguenza strutturale del formato: gzip comprime un flusso singolo e la sua signature sta nei primi byte; cosa contenga si scopre solo decomprimendo. **Nessuna regola FPR può separare i due casi**, perché il FPR ragiona per PUID e il PUID è lo stesso.

L'estrazione avviene però in due passaggi. Test su `doi:10.13130/RD_UNIMI/IHTWC0` (4 versioni, un `.tar.gz` da 17,9 MB ciascuna), METS dell'AIP risultante:

- formati: **4 GZip e 4 TAR** — uno per versione;
- percorsi annidati che documentano la catena: `STSWSN-PC_INSTANCES.tar-1.gz-<timestamp>/STSWSN-PC_INSTANCES.tar-<timestamp2>/STSWSN-PC_INSTANCES/graph100_8_2.txt` (prima la regola GZip `x-fmt/266`, poi la regola TAR `x-fmt/265`);

- **405 file Plain Text** estratti e identificati singolarmente.

Ne consegue che disabilitando la regola GZip **la catena si ferma al primo passo**: la regola TAR resta attiva ma non viene mai raggiunta, e i `.tar.gz` non vengono estratti affatto.

Il test ha mostrato anche il **fattore di moltiplicazione**: 16 file dichiarati dall'API hanno prodotto **465 oggetti** nell'AIP (29×).

10.4 Decisione adottata: configurazione differenziata a due lotti

Poiché nessuna regola può distinguere i due casi, la scelta è **differenziare per lotto** anziché per formato:

Fase	Regola GZip x-fmt/266	Dataset
Lotto generale	disabilitata	665 dataset: nessun blob inutile sugli ~11 500 <code>.rds</code> e <code>.fastq.gz</code>
Lotto dedicato agli archivi tar	riattivata	7 dataset: T99WYI, EIMQRQ, 3QA23K, IHTWC0, WHRHCT, BFQ87D, WCZXRK — estrazione completa in due passaggi

Motivazione da verbale: *l'estrazione dei file GZip a file singolo produce oggetti non utilizzabili e non identificabili, senza guadagno di conservazione, mentre l'oggetto autentico da preservare è il file compresso stesso. Poiché Siegfried non può distinguere un file singolo compresso da un archivio tar compresso (stesso PUID x-fmt/266), la regola di estrazione viene disabilitata per la lavorazione generale e riattivata per il solo lotto dei dataset che contengono veri archivi tar, verificati uno per uno.*

Questa soluzione non lascia nulla sul tavolo: i `.rds` restano intatti e i `.tar.gz` vengono comunque estratti.

Vincolo di sequenzialità. La regola FPR è globale: quando è riattivata vale per **qualunque** transfer in corso. Il lotto dedicato va quindi lavorato da solo, accertandosi che nessun altro lotto sia in lavorazione, e la regola va ridisabilitata subito dopo.

Da mettere a verbale: la configurazione FPR **non è stata uniforme** su tutto il corpus, ma differenziata consapevolmente per categoria di dataset. È una scelta legittima e documentabile, purché appunto documentata: senza questa nota, in sede di audit apparirebbe come un'incoerenza.

10.5 Applicazione e verifica

1. In FPR (*Preservation planning* -> *Format policy registry* -> *Rules*): **Replace** sulla regola GZip -> **Enabled: No**. **Disabilitare, non eliminare**: resta tracciata e la scelta è reversibile.
2. La modifica vale solo per i transfer **avviati dopo**: quelli in corso usano le regole lette in partenza.
3. Verifica sul dataset di test HADWPV (contiene un `.rds`): rimuovere la voce dal file di stato, rilanciare l'ingest e controllare che nel METS **non compaia più** l'evento `unpacking` e che gli oggetti scendano da 11 a 10.
4. Al momento del lotto dedicato: riattivare la regola, lavorare i 7 dataset, verificare la presenza di **GZip** e **TAR** tra i formati, quindi ridisabilitarla.
5. Gli AIP di prova prodotti prima della modifica vanno rifatti insieme al resto del corpus.

11. Export dei metadati per versione: limite noto dell'istanza

Durante i test è emerso che l'endpoint `/api/datasets/export` di `dataverse.unimi.it` **rifiuta il parametro version**: per ogni versione non corrente risponde 400 Bad Request.

```
HTTP 400 ... /api/datasets/export?exporter=dataverse_json&persistentId=...&version=1.0
[ATTENZIONE] Export dataverse_json non riuscito
```

Su un dataset con 4 versioni, gli export riescono solo per l'ultima (v1.3) e falliscono per v1.0, v1.1, v1.2. È la ragione tecnica per cui `schema.json` è presente solo nella cartella dell'ultima versione (cfr. manuale).

Conseguenza operativa da valutare prima del lotto completo: lo script esegue **3 tentativi con retry** per ogni export fallito, cioè 6 chiamate inutili per versione (due exporter). Sulle 1 226 versioni dello scope sono diverse migliaia di richieste sprecate, che allungano i tempi e caricano inutilmente il server.

Poiché un **400** è un errore **definitivo** e non transitorio, non andrebbe ritentato. La correzione è circoscritta a `scarica_dataverse.py` e conviene applicarla prima della lavorazione estesa.

12. Piano dei lotti e calibrazione

Lo scope si suddivide con `prepara_lotti.py`, che legge il report di Fase 0 e bilancia i lotti su **numero di file e volume**, non sul conteggio dei DOI: il corpus è estremamente disomogeneo (da 1 file a quasi 15 000 per dataset) e lotti “da N DOI” avrebbero durate imprevedibili.

```
python3 prepara_lotti.py --max-file 2000 --max-gb 20 --max-doi 200 --scrivi
```

Con queste soglie lo scope dei 672 DOI si ripartisce in **27 lotti** (67 730 file, 62,95 GB). La struttura riflette due popolazioni distinte:

- **lotti 01-20:** dominati dal numero di file, quasi tutti un solo DOI (il maggiore ha 14 888 file, il 22% dei file dell'intero corpus);
- **lotti 21-27:** dominati dal numero di pacchetti (511 DOI per ~2 100 file), dove il costo è quasi tutto overhead per transfer.

Un solo file di stato per l'intera ri-acquisizione (`stato_riacquisizione.json`): è ciò che impedisce doppi ingest se un DOI comparisse in due lotti o se un lotto venisse rilanciato.

12.1 Dati di calibrazione misurati

Dal primo lotto reale (111 DOI, 118 file, 2 GB — quasi solo overhead):

Indicatore	Valore
Pacchetti completati	110 su 111
Tempo totale ingest	75,5 min
Media per pacchetto	41,6 s (mediana 40 s, min 25 s, max 251 s)

Il tempo per pacchetto è dominato dall'attesa di approvazione (10 s) e dal polling (intervalli di 15 s), quindi è **indipendente dalla dimensione del pacchetto**. Estrapolando sui 672 pacchetti: circa **7,8 ore di solo overhead**, a cui si somma il lavoro effettivo sui dataset con molti file.

Ne discende che alzare le soglie per lotto **non allunga i tempi totali** (i pacchetti restano 672): riduce solo il numero di cicli manuali. Se le ore di polling diventassero un problema, l'unica leva è ridurre gli intervalli in `archivematica_ingest.py`, valutando però il carico sulla Dashboard.

Resta da misurare il **costo per file** dentro un pacchetto, con un lotto a pacchetto singolo e molti file (es. 1 DOI / ~1 000 file): con i due coefficienti si può stimare ogni riga del piano, incluso il lotto oversize.

13. Criteri di GO / NO-GO per il lotto completo

Si procede al lotto dei 702 dataset **solo se**:

- entrambi i Gate del pilota sono superati senza eccezioni;
- in particolare il Pilota B dimostra che i file **ristretti e ingeriti** vengono scaricati come originali (nessun [FALLBACK], conteggio file corretto);

- è stata presa una decisione **strutturale** sulla collocazione di `TRANSFER_SOURCE`: 63 GB in 67 730 file dentro una cartella sincronizzata con Dropbox riproporrebbero i lock transitori su larga scala. La sospensione manuale della sincronizzazione non è sufficiente per una lavorazione lunga, perché Dropbox può riprendere dopo un riavvio o un aggiornamento del client;
- lo spazio disponibile copre 63 GB di download più lavorazione e AIP;
- è stato definito il **dimensionamento dei lotti** (per esempio 50–100 DOI per volta) e il punto di ripresa in caso di interruzione. I lotti si organizzano in **sottocartelle della Transfer Source Location** (`LOTTO_01`, `LOTTO_02`...), da indicare con `--source`: è verificato che Archivematica le risolva correttamente. Omettere `--source` farebbe usare la radice della location, con il rischio di rilavorare tutti i pacchetti presenti;
- la **processing configuration** è quella corretta (prerequisito 4.7): un errore qui obbligherebbe a rifare tutti gli AIP del lotto;
- la **politica di trattamento degli archivi** è stata decisa e applicata (sezione 10), applicata e verificata con il test del `.rds`: modificarla a metà lavorazione produrrebbe un corpus disomogeneo, con alcuni AIP contenenti blob estratti e altri no;
- il **censimento degli archivi** (`censisci_archivi.py`) è completo, senza DOI rimasti non analizzati: i dataset che contengono archivi hanno un carico reale molto superiore a quello dichiarato dall'API e vanno collocati nei lotti di conseguenza;
- i **7 dataset con archivi tar** sono stati estratti dal piano generale e raccolti in un **lotto dedicato** da lavorare separatamente con la regola GZip riattivata (sezione 10.4).

Nota a favore: la ri-acquisizione è ripartibile in sicurezza. Lo skip si basa su dimensione e checksum, quindi rilanciare non lascia file monchi; un download interrotto viene ri-scaricato al passaggio successivo.

14. Registrazione dell'esito

Al termine del pilota annotare, per ciascun dataset: data di esecuzione, esito dei due Gate, UUID dell'AIP prodotto, anomalie riscontrate e correzioni applicate. Questo verbale è la base documentale della ricostruzione del corpus e va conservato insieme alla documentazione del progetto.

14.1 Esito del pilota del 22 luglio 2026

Gate 1 superato su entrambi i dataset: 36 file scaricati (10 + 26), di cui 31 come originali (`format=original`) e 5 non ingeriti; nessun `[FALLBACK]`, nessun `.tab` su disco, nessun errore o riscarico. Le 4 versioni di PKZ7FJ e le 3 di DP08SC complete. Le 36 `dcats:distribution` coerenti: 31 con `?format=original`, 5 senza (file non ingeriti).

Gate 2 superato: formati identificati da Siegfried 1.11.2 (25 Microsoft Excel su DP08SC, più PDF, CSV, JSON, Plain Text), nessun Unknown, nessun Tab-Separated Values. Su PKZ7FJ 2 eventi di `normalization` con 2 derivate PDF/A validate. I 25 nomi con spazi risultano sanificati sul filesystem e documentati in `originalName` con relativi eventi `filename change`.

Anomalie riscontrate e risolte durante il pilota:

1. *Ingest fallito con i pacchetti in sottocartella* (`Filepath ... does not exist`): bug di `archivematica_ingest.py`, che ricostruiva il percorso dal solo nome della cartella perdendo la sottocartella. Corretto passando il path assoluto del pacchetto; verificato con un transfer completato da sottocartella.
2. *Tutti i formati Unknown al primo ingest*: identificazione dei formati disattivata nella `processing configuration` (vedi prerequisito 4.7). Corretta e ingest ripetuto.
3. *Permessi in scrittura sulla Transfer Source*: il mount `drvfs` usa `uid=333;gid=333;umask=022`, quindi al gruppo manca il bit `w`. Risolto con `chmod g+ws` sulla cartella; per il lotto va valutata una soluzione stabile (`umask=002` in `/etc/wsl.conf` o riposizionamento della location).

14.2 Esito della calibrazione sul primo lotto reale (LOTTO_27)

111 DOI leggeri (118 file, 2 GB): **110 ingeriti, 1 fallito**, 75,5 minuti, media 41,6 s per pacchetto (vedi 12.1).

Il caso fallito — un dataset composto da un unico archivio compresso da 610 MB — ha fatto emergere tre problemi a catena, tutti documentati nella sezione 9: la domanda *Extract packages?* non coperta dalla configurazione (9.1), il disallineamento di MCPClient dopo il riavvio di MCPServer (9.2) e il riaggancio di un transfer annullato rimasto nel file di stato (9.3).

Lezione per il lotto completo: un solo caso su 111 è circa lo 0,9%, che sui 672 pacchetti significherebbe una sessantina di transfer bloccati. Il censimento preventivo con `censisci_archivi.py` ha poi individuato **39 dataset con archivi compressi** (12 043 archivi, 11,88 GB), portando alla revisione della politica di estrazione documentata nella sezione 10.

Da questo caso è emerso anche che il piano dei lotti **sottostima sistematicamente** i dataset contenenti archivi: WPOXS5 risultava «1 file, 0,58 GB» e ha prodotto **2 238 elementi** nell'AIP (1 859 eventi di unpacking), perché conteneva un archivio NMR con una gerarchia profonda 10 livelli. I lotti che contengono dataset con archivi vanno quindi considerati più pesanti di quanto il conteggio dei file suggerisca.